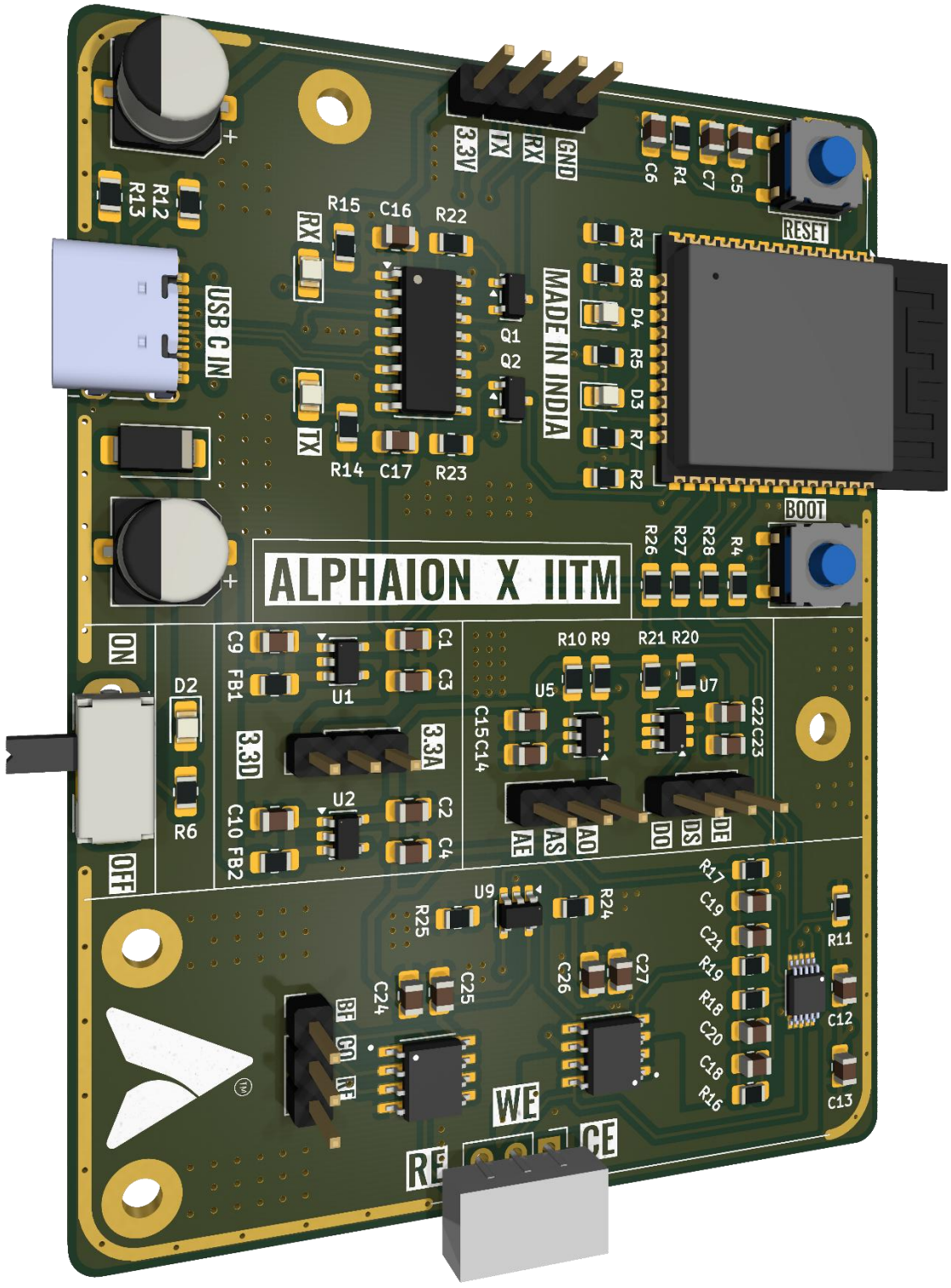


ALPHION POTENTIOSTAT

Technical Documentation

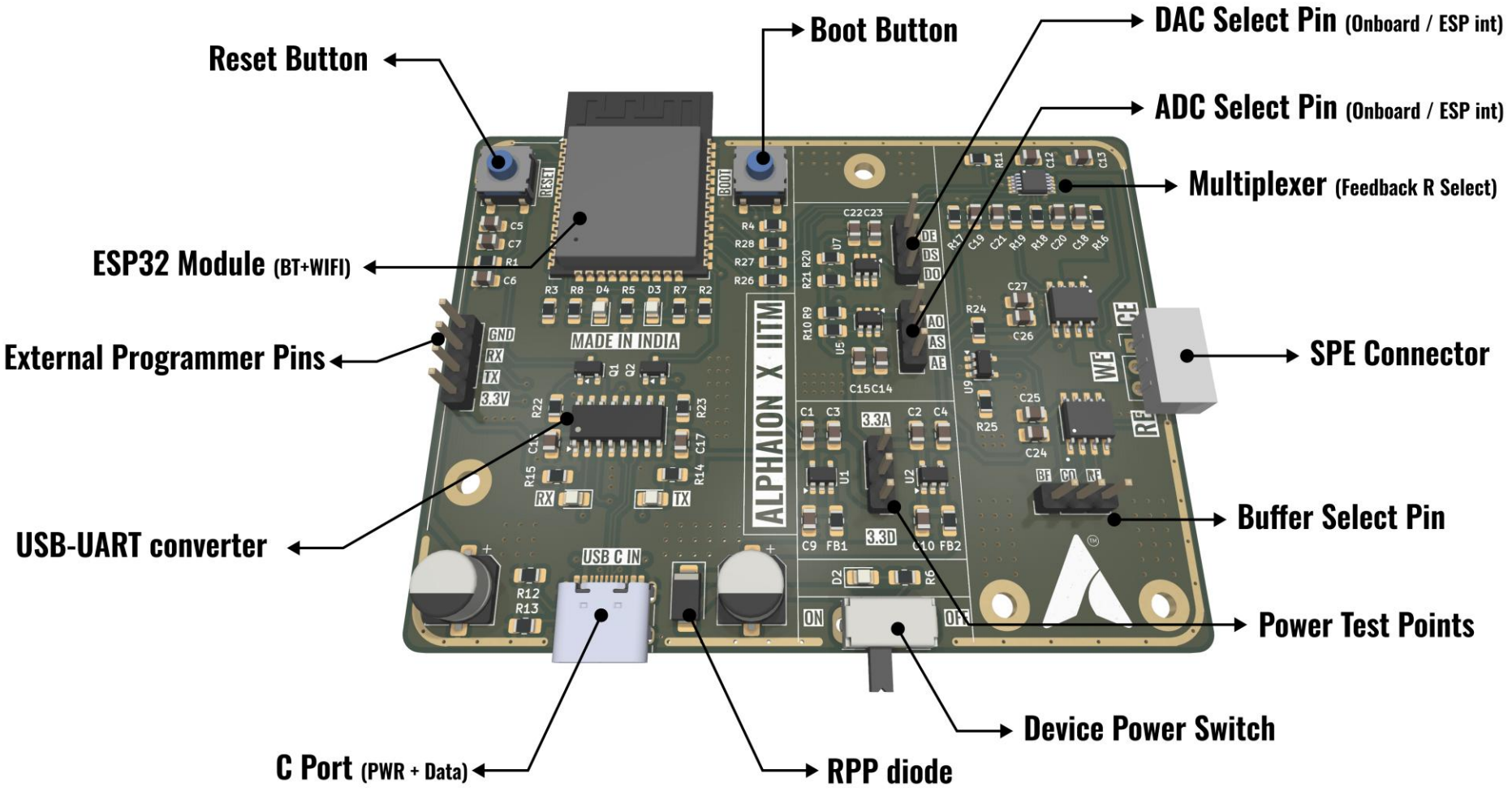
Rev 1 | 2026

Contents

Flashing and Data Transfer using external Dev board 4
Checking ADC & DAC Address 5
Checking Working of ADC MCP3421 (using ESP32 DAC) 5
Checking Working of ADC MCP7425 (using ESP32 ADC) 6
Checking Working of ADC MCP7425 and DAC MCP3421 7
Checking for Resistance 8

List of Figures

Figure 1:- Connection for flashing the board using external ESP32 Dev Module 4
Figure 2:- Serial Monitor Output for the address 5
Figure 3:- Connecting the ESP32 DAC to On-board ADC 6
Figure 4:- Expected Output of the MCP3421 ADC test 6
Figure 5:- Connecting the ESP32 ADC to On-board DAC 6
Figure 6:- Expected Output of the MCP4725 DAC test 7
Figure 7:- Connecting the On-board ADC to On-board DAC 7
Figure 8:- Expected Output of the MCP4725 DAC MCP3421 ADC syn test 8
Figure 9:- Connecting the Resistor to the electrode slot 8
Figure 10:- Expected Output of the 10K ohm Resistor test 9
Figure 11:- Expected Output of 5K ohm Resistor test 9
Figure 12:- IV curve for different resistors with different slant angle 9


Reset Button
Boot Button
DAC Select Pin (Onboard / ESP int)
ADC Select Pin (Onboard / ESP int)
Multiplexer (Feedback R Select)
ESP32 Module (BT+WIFI)
External Programmer Pins
SPE Connector
USB-UART converter
Buffer Select Pin
Power Test Points
C Port (PWR + Data)
RPP diode
Device Power Switch
RESET
BOOT
GND
RX
TX
3.3V
MADE IN INDIA
ALPHAION X IITM
USB C IN
ON
OFF
3.3A
3.3D
DE
DS
DO
AO
AS
AE
WE
RF

Flashing and Data Transfer using external Dev board

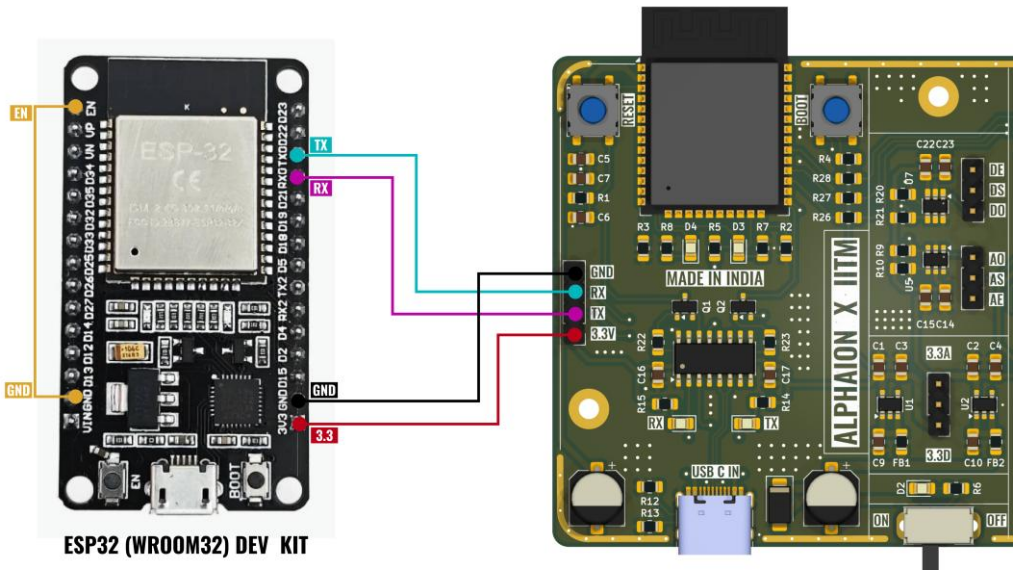


Figure 1:- Connection for flashing the board using external ESP32 Dev Module

Step 1: Make the Hardware Connections

- Connect the external ESP32 DevKit to the onboard ESP32 through the UART interface.

External ESP32 DevKit	Onboard ESP32
TX0 (GPIO1)	RX0 (GPIO3)
RX0 (GPIO3)	TX0 (GPIO1)
GND	GND

- Ensure that TX and RX connections are crossed and that both boards share a common ground.

Step 2: Disable the External ESP32

- Connect the EN pin of the external ESP32 DevKit to GND.
- This disables the external ESP32 microcontroller while keeping its USB-to-UART converter operational.

Step 3: Connect the External DevKit to the Computer

- Connect the external ESP32 DevKit to the computer using a USB cable.
- The external DevKit's USB-to-UART converter will provide the serial communication interface required for firmware uploading.

Step 4: Configure the Arduino IDE

- Open Arduino IDE and select the following settings:

Parameter	Configuration
Board	ESP32 Dev Module
Port	External DevKit COM port
Upload Speed	115200

Step 5: Enter the Onboard ESP32 into Bootloader Mode

- Follow this sequence:

    - Press and hold the BOOT button on the onboard PCB.
    - Press and release the RESET button.
    - Release the BOOT button.
- The onboard ESP32 is now in bootloader mode and ready to receive firmware.

Step 6: Upload the Firmware

- Open the required Arduino sketch and click the Upload button.
- The firmware will be transferred from the computer through the external DevKit's USB-to-UART converter to the onboard ESP32.

Step 7: Restart the Onboard ESP32

- After successful uploading:
    - Press the RESET button on the onboard PCB.
    - Ensure that the onboard PCB is powered appropriately.
    - The ESP32 will restart and execute the newly uploaded firmware.

For subsequent wired serial communication, retain the required UART connections and ensure that the onboard and external USB-to-UART transmitters do not conflict.

Checking ADC & DAC Address

- Upload the Code ***ADC_DAC_TEST.ino*** code to the board
- Open the serial Monitor
- Press the reset button
- Check for the ADC and DAC addresses

```
07:15:14.596 -> ================================
07:15:14.596 -> POTENTIOSTAT I2C SCANNER
07:15:14.596 -> ================================
07:15:14.596 ->
07:15:14.596 -> Scanning DAC BUS (MCP4725)
07:15:14.596 -> Device found at address: 0x60
07:15:14.596 -> Total devices found: 1
07:15:14.596 ->
07:15:14.596 -> Scanning ADC BUS (MCP3421)
07:15:14.645 -> Device found at address: 0x68
07:15:14.645 -> Total devices found: 1
07:15:14.645 ->
07:15:14.645 -> I2C SCAN COMPLETED
```

Figure 2:- Serial Monitor Output for the address

Checking Working of ADC MCP3421 (using ESP32 DAC)

- On the board short the **DE** and **AO** pins

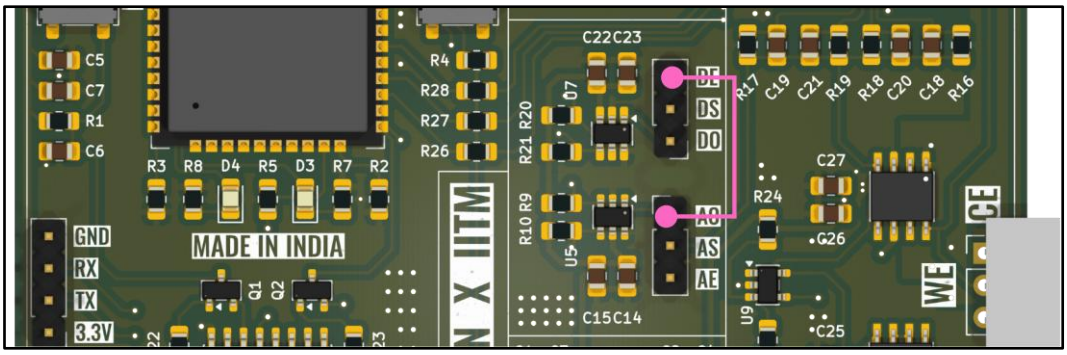


Figure 3:- Connecting the ESP32 DAC to On-board ADC

- Upload the Code ***MCP3421_TEST.ino*** code to the board
- Open the Python Script ***MCP3421_TEST.py***
- Check for the COM PORT number in the device manager
- Change the COM PORT value in the Python Code
- Run the python script once the code is uploaded.
- Additionally to check the ADC values disconnect the connection of step 1 to see a flat line in the graph.

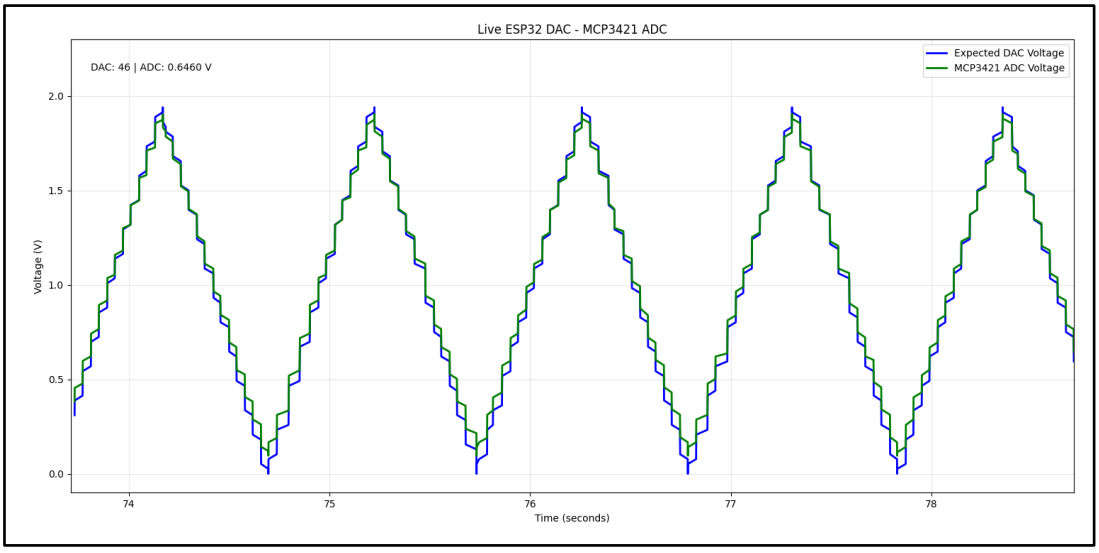


Figure 4:- Expected Output of the MCP3421 ADC test

Checking Working of ADC MCP7425 (using ESP32 ADC)

- On the board short the **DO** and **AE** pins

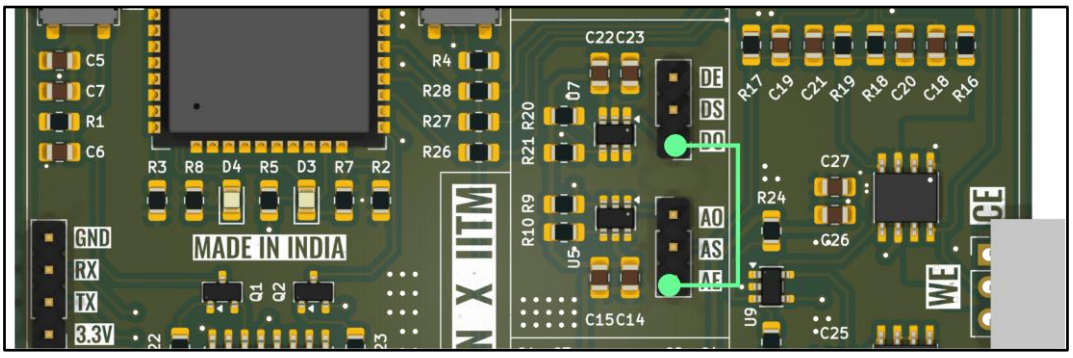


Figure 5:- Connecting the ESP32 ADC to On-board DAC

- Upload the Code ***MCP7425_TEST.ino*** code to the board
- Open the Python Script ***MCP7425_TEST.py***
- Check for the COM PORT number in the device manager

- Change the COM PORT value in the Python Code
- Run the python script once the code is uploaded.
- Additionally to check the DAC values disconnect the connection of step 1 to see a flat line in the graph.

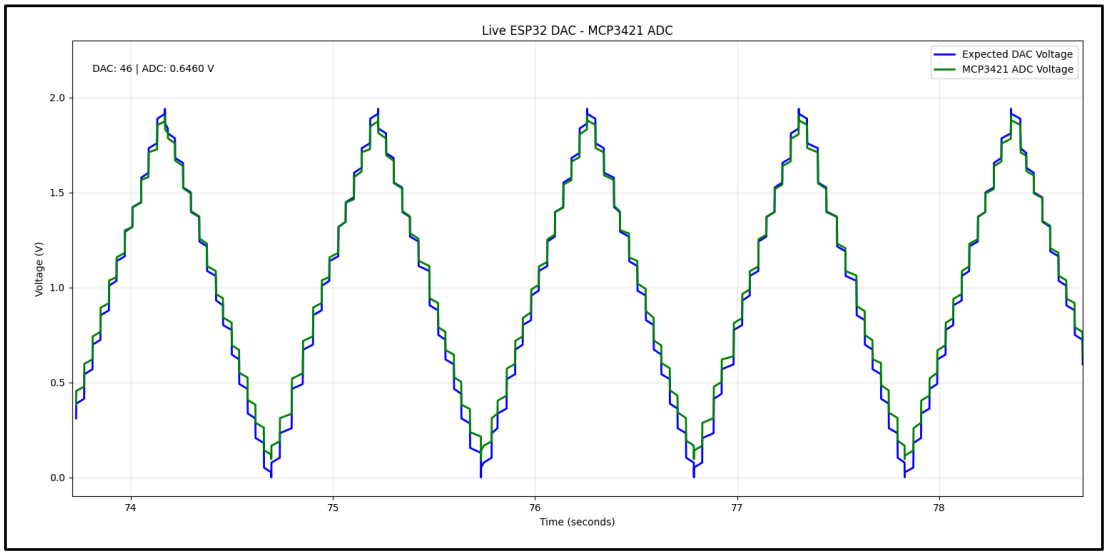


Figure 6:- Expected Output of the MCP4725 DAC test

Checking Working of ADC MCP7425 and DAC MCP3421

- On the board short the **DO** and **AO** pins

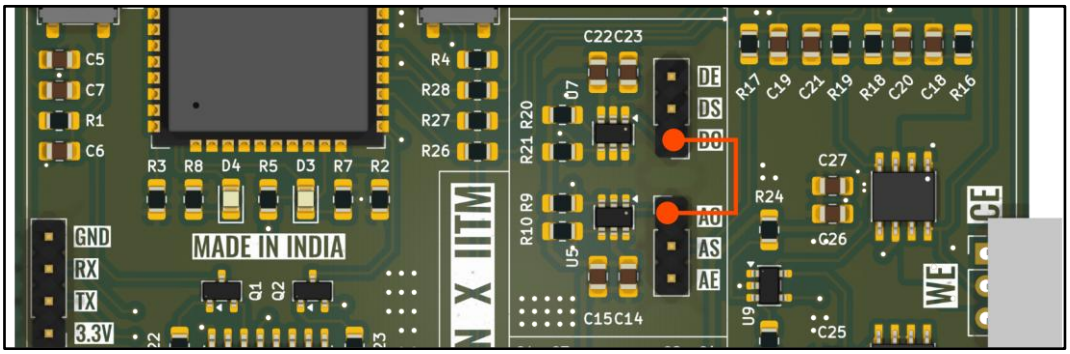


Figure 7:- Connecting the On-board ADC to On-board DAC

- Upload the Code ***MCP7425_ MCP3421_SYNC_TEST.ino*** code to the board
- Open the Python Script ***MCP7425_ MCP3421_SYNC_TEST.py***
- Check for the COM PORT number in the device manager
- Change the COM PORT value in the Python Code
- Run the python script once the code is uploaded.
- Additionally to check the DAC values disconnect the connection of step 1 to see a flat line in the graph.

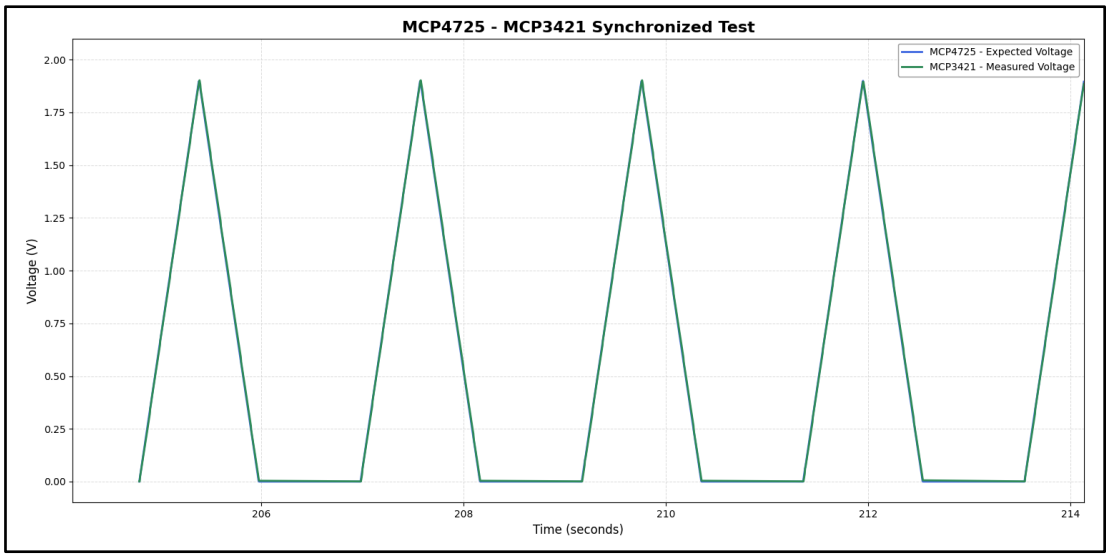


Figure 8:- Expected Output of the MCP4725 DAC MCP3421 ADC syn test

Checking for Resistance

- On the board short the **DO & DS**, **AO & AS and BF & CO** pins
- Connect the **CE & RE** to one end of Resistor and **WE** to other end

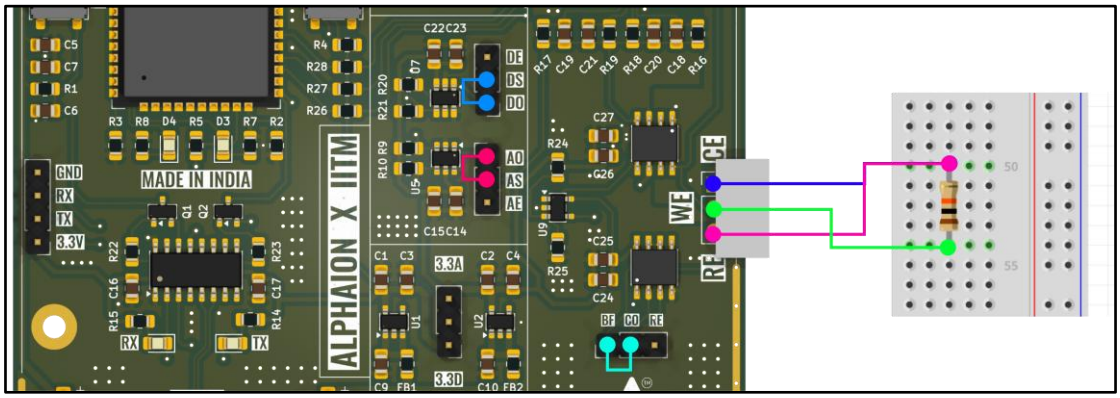


Figure 9:- Connecting the Resistor to the electrode slot

- Upload the Code ***RESISTOR_TEST.ino*** code to the board
- Open the Python Script ***RESISTOR_TEST py***
- Check for the COM PORT number in the device manager
- Change the COM PORT value in the Python Code
- Run the python script once the code is uploaded.
- Additionally to check the Resistance values connect 10 K ohm in parallel to the existing resistor and see the plot falling to 5000.

Second Test IV curve

- Open the python script **RESISTOR_TEST_IV.py**
- Check for the COM PORT number in the device manager
- Change the COM PORT value in the Python Code
- Run the python script once the code is uploaded.
- Additionally to check the Resistance values connect different known resistor and check the plot angle changing.

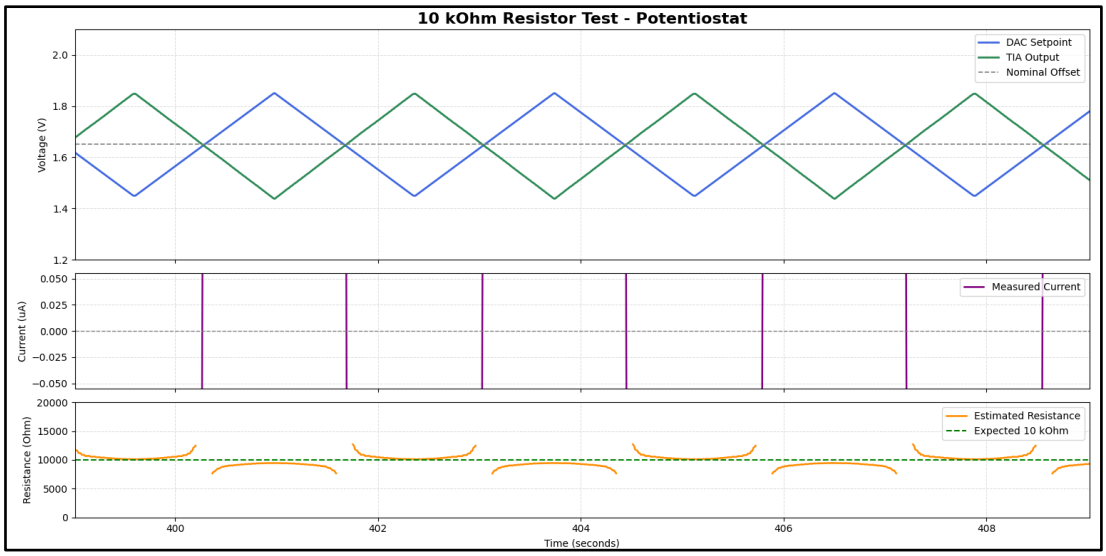


Figure 10:- Expected Output of the 10K ohm Resistor test

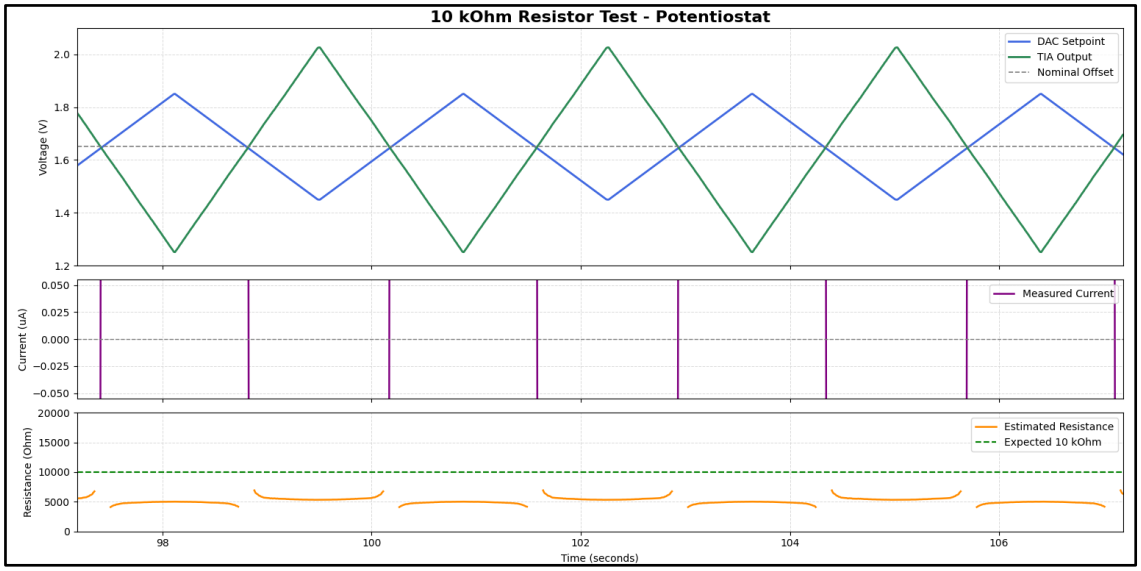


Figure 11:- Expected Output of 5K ohm Resistor test

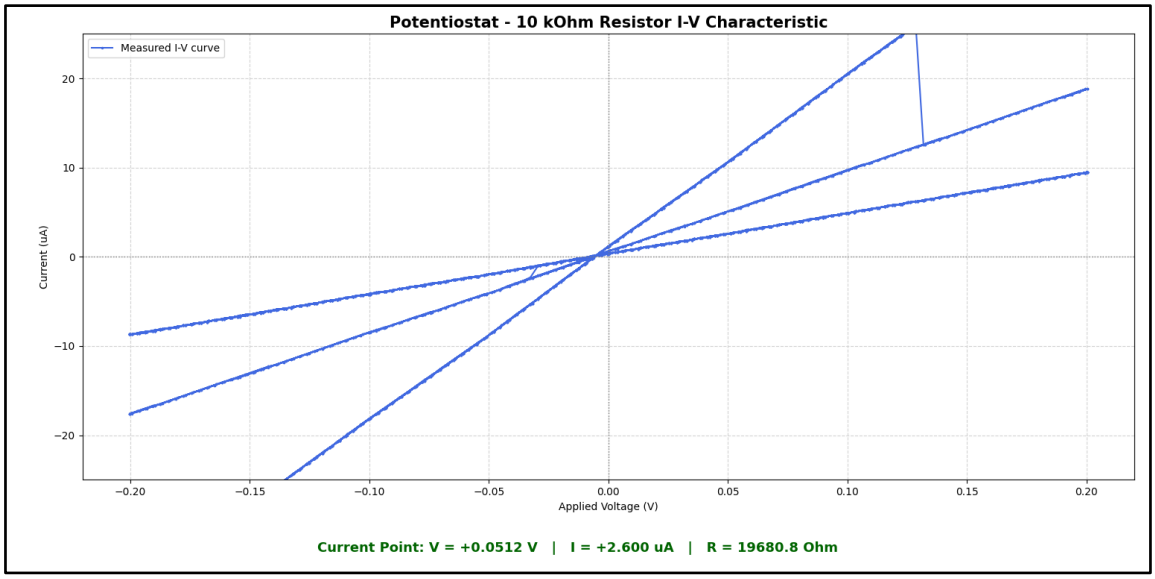


Figure 12:- IV curve for different resistors with different slant angle